TASK 1

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 Node* temp = head;
 int count = 0;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 count++;
 }
 cout << "NULL";
 cout << "\nTotal Nodes: " << count;
 return 0;
}
```

Task 2: Insert a Node

2.1 Insert at Beginning

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 Node* newNode = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 newNode->data = 5;
 newNode->next = head;
 head = newNode;
 Node* temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```

2.2 Insert at End

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 Node* newNode = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 newNode->data = 40;
 newNode->next = NULL;
 third->next = newNode;
 Node* temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```



2.3 Insert at Specific Position

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 Node* newNode = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 newNode->data = 25;
 newNode->next = third;
 second->next = newNode;
 Node* temp = head;
while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```



Task 3: Delete a Node

3.1 Delete from Beginning

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 Node* temp = head;
 head = head->next;
 delete temp;
 temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```



3.2 Delete from End

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 second->next = NULL;
 delete third;
 Node* temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```

Tab to Jump

3.3 Delete from Specific Position

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 second->next = NULL;
 delete third;
 Node* temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```

Task 4: Search an Element

nn.cc

```cpp
#include <iostream>
using namespace std;
struct Node
{
 int data;
 Node* next;
};
int main()
{
 Node* head = new Node;
 Node* second = new Node;
 Node* third = new Node;
 head->data = 10;
 second->data = 20;
 third->data = 30;
 head->next = second;
 second->next = third;
 third->next = NULL;
 second->next = NULL;  Tab to Jump
 delete third;
 Node* temp = head;
 while (temp != NULL)
 {
 cout << temp->data << " -> ";
 temp = temp->next;
 }
 cout << "NULL";
 return 0;
}
```

Task 5: Conceptual Questions

1. **Singly Linked List:** A data structure where each node contains data and a pointer to the next node.
   Array: Stores elements in continuous memory, while linked list nodes can be anywhere in memory.
2. A node has two main parts: **Data** and **Next pointer**.
3. A pointer is required to **connect one node to the next node**.
4. The next pointer of the last node contains **NULL**.
5. **Head** points to the first node, while a normal pointer can point to **any node**.
6. Because insertion at the beginning only requires **changing the head pointer**, so it takes **O(1)** time.
7. If head == NULL, the linked list is **empty**.
8. Traversal is required because linked lists **do not support direct indexing** like arrays.
9.

- Traversal → **O(n)**
- Searching → **O(n)**
- Insertion at beginning → **O(1)**
- Deletion at beginning → **O(1)**

10. If nodes are not deleted properly, it can cause **memory leaks**, wasting memory and potentially slowing or crashing the program.